Quest

February 2026

1 Simulations

We consider a Stochastic Differential Equation (SDE) of the form:

$$dX(t) = f(X(t))\,dt + g(X(t))\,dW(t), \quad 0 \leq t \leq T \tag{1}$$

with the initial condition $X(0) = X_0$.

1.1 Numerical Discretization

To simulate the path of $X(t)$, we partition the interval $[0, T]$ into N equal steps of size $\Delta t = T/N$, where $t_i = i\Delta t$. The **Euler-Maruyama** approximation is given by:

$$X_{n+1} = X_n + f(X_n)\Delta t + g(X_n)\Delta W_n \tag{2}$$

where the Brownian increment is defined as:

$$\Delta W_n = W(t_{n+1}) - W(t_n) \approx \sqrt{\Delta t} \cdot \mathcal{N}(0,1) \tag{3}$$

Here, $\mathcal{N}(0,1)$ represents a standard normal random variable.

2 Python Implementation

The following Python script implements the Euler-Maruyama discretizations to simulate a standard Brownian Motion path.

Listing 1: Python code for Brownian Motion

```python
import numpy as np
import matplotlib.pyplot as plt

T = 1.0          # total time
N = 1000         # number of steps
dt = T / N

t = np.linspace(0, T, N)
X = np.zeros(N)

for i in range(1, N):
    Z = np.random.normal(0, 1)
    X[i] = X[i-1] + np.sqrt(dt) * Z

plt.plot(t, X)
plt.title("Brownian␣Motion␣Simulation␣(Euler-Maruyama)")
plt.xlabel("Time")
plt.ylabel("X(t)")
plt.show()
```

3 Simulation Results

The resulting path of the stochastic process is visualized in Figure 1.

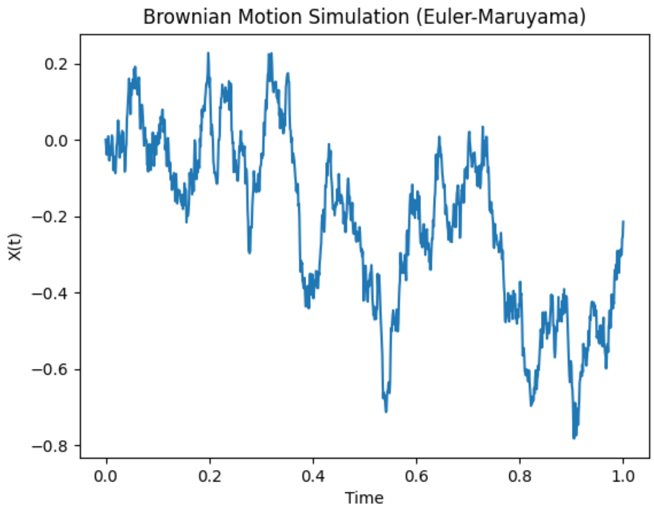

Figure 1: Numerical simulation of Brownian Motion using $N = 1000$ steps.

4 Stock Price Simulation (Euler-Maruyama)

While the Black-Scholes SDE has an analytical solution, we can approximate the stock price path numerically using the Euler-Maruyama method. The discrete-time approximation for the price S at step i is given by:

$$S_i = S_{i-1} + \mu S_{i-1}\Delta t + \sigma S_{i-1}\sqrt{\Delta t}Z_i \tag{4}$$

where $Z_i \sim \mathcal{N}(0, 1)$ is a standard normal random variable.

4.1 Python Implementation

Below is the Python code used to generate the simulation for an initial price $S_0 = 100$, drift $\mu = 0.1$, and volatility $\sigma = 0.2$.

Listing 2: Linear Euler-Maruyama for Stock Prices

```
import numpy as np
import matplotlib.pyplot as plt

T = 1
N = 1000
```

```
dt = T/N

mu = 0.1
sigma = 0.2
S0 = 100

S = np.zeros(N)
S[0] = S0

for i in range(1, N):
    Z = np.random.normal()
    S[i] = S[i-1] + mu*S[i-1]*dt + sigma*S[i-1]*np.sqrt(dt)*
        Z

plt.plot(S)
plt.title("Stock Price Simulation (Euler-Maruyama)")
plt.xlabel("Time Steps")
plt.ylabel("Stock Price")
plt.show()
```

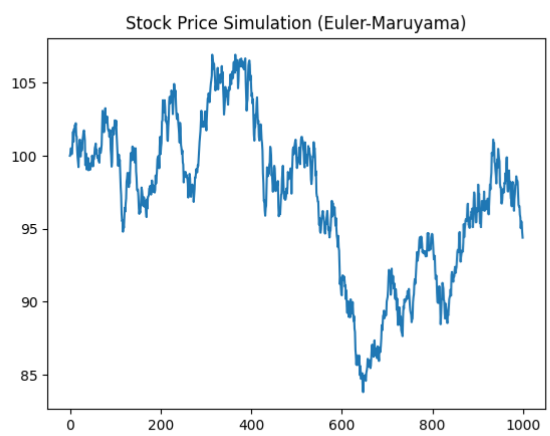

Figure 2: Simulated stock price path showing stochastic drift and volatility.

4.2 Intuition and Accuracy

The core intuition behind the Euler-Maruyama discretization for stock prices is:

Price after an instant = *Current Price* + *Drift* + *Diffusion (Randomness)*

This captures the two main drivers of market movement: the long-term trend (μ) and the unpredictable noise (σ).

We now directly compare the path simulated using the Euler-Maruyama method against the exact analytical solution of the geometric Brownian Motion SDE. This visualization allows us to assess the accuracy of the discretization.

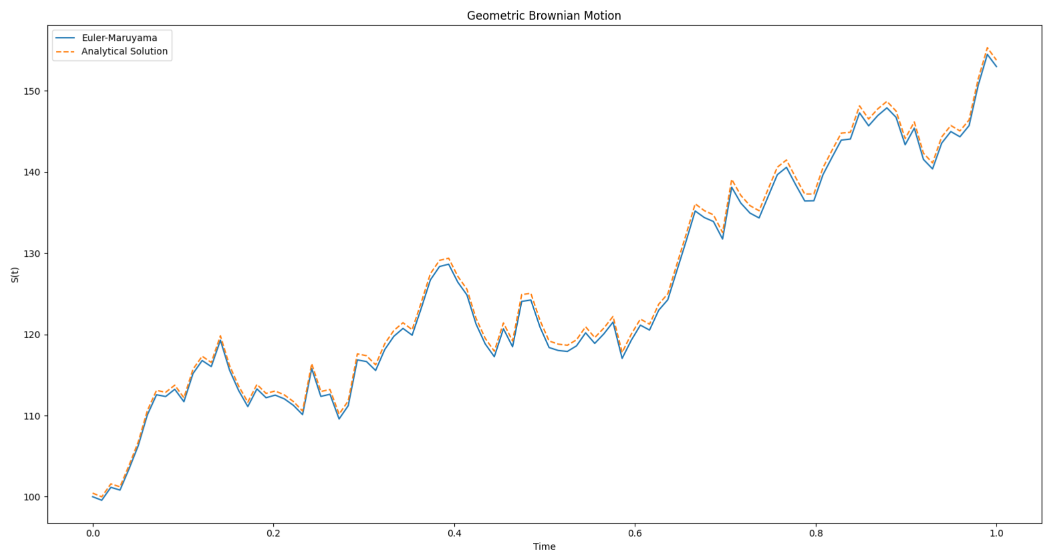

Figure 3: Close-up comparison of the Analytical Solution and the Euler-Maruyama Numerical Approximation. Note how closely the numerical scheme tracks the analytical path with $N = 1000$ steps.

Note on Convergence: As we increase the number of steps N (and thus decrease Δt), the numerical solution converges toward the analytical solution. Therefore, the accuracy of the simulation increases as $N \to \infty$.

4.3 Convergence Rates

The Euler-Maruyama method exhibits a **strong convergence** order of $\gamma = 0.5$ and a **weak convergence** order of $\beta = 1.0$. This implies that while the individual paths may vary, the expected value of the solution converges linearly with the time step.

5 Stability and the Positivity Constraint

In the continuous Black-Scholes model, the stock price S_t follows a log-normal distribution and remains strictly positive ($S_t > 0$). However, the linear Euler-

Maruyama discretization:

$$S_i = S_{i-1} + \mu S_{i-1}\Delta t + \sigma S_{i-1}\Delta W_i \tag{5}$$

can theoretically produce negative prices if the volatility σ is high or the time step Δt is too large, as a large negative shock ΔW_i could overwhelm the current price.

Insight: To guarantee positivity, practitioners often simulate the log-price $Y_t = \ln(S_t)$ using Itô's Lemma:

$$dY_t = \left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma dW_t \tag{6}$$

And then transform back via $S_t = e^{Y_t}$.

6 Monte Carlo Pricing

The power of the Euler-Maruyama method lies in its application to **Monte Carlo Integration**. To find the fair value of a European Call Option today (V_0), we simulate M independent paths of the stock price until maturity T.

The value is the discounted average of the payoffs:

$$V_0 \approx e^{-rT} \left(\frac{1}{M} \sum_{j=1}^{M} \max(S_T^{(j)} - K, 0) \right) \tag{7}$$

Where:

- M is the total number of simulations.
- K is the strike price.
- r is the risk-free interest rate.

7 Formal Definition of Brownian Motion

A stochastic process $\{W_t\}_{t\geq 0}$ is called a **Standard Brownian Motion** (or a **Wiener Process**) if it satisfies the following four conditions:

1. **Initial Condition:** The process starts at zero:

$$W_0 = 0$$

2. **Independent Increments:** For any partition of time $0 \leq t_1 < t_2 < \cdots < t_n$, the increments

$$W_{t_2} - W_{t_1}, W_{t_3} - W_{t_2}, \ldots, W_{t_n} - W_{t_{n-1}}$$

are independent random variables.

3. **Gaussian Increments:** For any $0 \le s < t$, the increment $W_t - W_s$ is normally distributed with mean 0 and variance $t - s$:

$$W_t - W_s \sim \mathcal{N}(0, t - s)$$

This implies that the probability density function of W_t is given by:

$$f(x,t) = \frac{1}{\sqrt{2\pi t}} \exp\left(-\frac{x^2}{2t}\right)$$

4. **Continuity of Paths:** The function $t \mapsto W_t$ is continuous in t.

8 Properties and Path Behavior of Wiener Process

- **Quadratic Variation:** Unlike smooth functions, Brownian motion has non-zero quadratic variation. For a partition of $[0, t]$:

$$[W, W]_t = \lim_{\|\Pi\| \to 0} \sum (W_{t_{i+1}} - W_{t_i})^2 = t \text{ (a.s.)}$$

- **Non-Differentiability:** Paths are *continuous everywhere* but *differentiable nowhere.*

- **Martingale Property:** The process is driftless. Given the filtration $\mathcal{F}_s$:

$$\mathbb{E}[W_t \mid \mathcal{F}_s] = W_s, \quad \text{for } t \ge s$$

9 First and Second Order Moments

For a standard Brownian motion $\{B_t\}_{t \ge 0}$ the following stochastic properties hold:

- **Mean:** Since $B_t \sim \mathcal{N}(0, t)$, the first moment is:

$$\mathbb{E}[B_t] = 0, \quad \forall t \ge 0$$

- **Variance:** The second central moment is linear in t:

$$\mathrm{Var}(B_t) = \mathbb{E}[B_t^2] - (\mathbb{E}[B_t])^2 = t$$

This is why it follows quadratic variation, since the variance scales linearly. This ties into Ito's Lemma.

- **Autocovariance Function:** For $s, t \geq 0$, the covariance is given by the minimum of the time indices. Without loss of generality, assume $s \leq t$:

$$\begin{aligned}
\mathrm{Cov}(B_s, B_t) &= \mathbb{E}[B_s B_t] - \mathbb{E}[B_s]\mathbb{E}[B_t] \\
&= \mathbb{E}[B_s(B_t - B_s + B_s)] \\
&= \mathbb{E}[B_s(B_t - B_s)] + \mathbb{E}[B_s^2] \\
&= \mathbb{E}[B_s]\mathbb{E}[B_t - B_s] + s \\
&= 0 \cdot 0 + s = \min(s, t)
\end{aligned}$$

10 Convergence of Random Walks and Universal Behavior

The Donsker's Invariance Principle generalizes the Central Limit Theorem from independent variables to entire stochastic processes. It states that a suitably rescaled discrete-time random walk converges in distribution to a standard Brownian motion B_t as the number of steps n tends to infinity.

Let $\{X_i\}_{i=1}^{\infty}$ be a sequence of independent and identically distributed (i.i.d.) random variables with mean $\mathbb{E}[X_i] = 0$ and finite variance $\mathrm{Var}(X_i) = \sigma^2$. The discrete random walk is defined as $S_n = \sum_{i=1}^{n} X_i$. For continuous time $t \in [0, 1]$, the scaled process is defined as:

$$W_n(t) = \frac{1}{\sigma\sqrt{n}} S_{\lfloor nt \rfloor}$$

10.1 Verification of Convergence via Simulation

To illustrate this universal convergence, we compare the sample paths and the resulting terminal distributions of two distinct random walks with $n = 1000$ steps and 5000 simulations. The first case uses discrete Bernoulli increments $X_i \in \{-1, 1\}$, and the second uses continuous Gaussian increments $X_i \sim \mathcal{N}(0, 1)$.

10.1.1 Path Convergence

The simulated trajectories demonstrate that despite the Bernoulli walk being limited to discrete steps of $\pm 1/\sqrt{n}$, its macroscopic behavior as n becomes large is indistinguishable from the continuous Gaussian random walk. The scaling factor $1/\sqrt{n}$ is essential for preserving the finite variance structure required by the limit process.

11 Brownian Motion with Drift

Brownian motion with drift $\{X_t\}_{t \geq 0}$ incorporates a deterministic linear trend into the stochastic process. This model is formally expressed as:

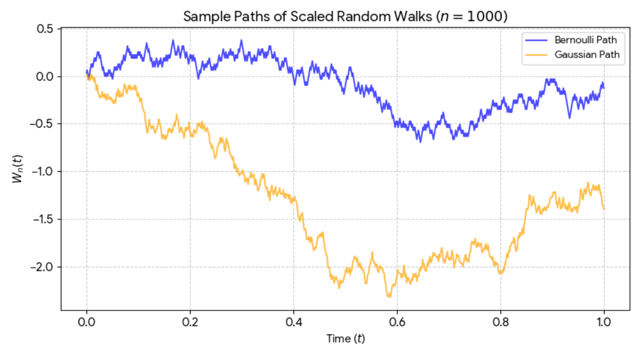

Figure 4: Comparison of sample paths for scaled Bernoulli and Gaussian random walks ($n = 1000$).

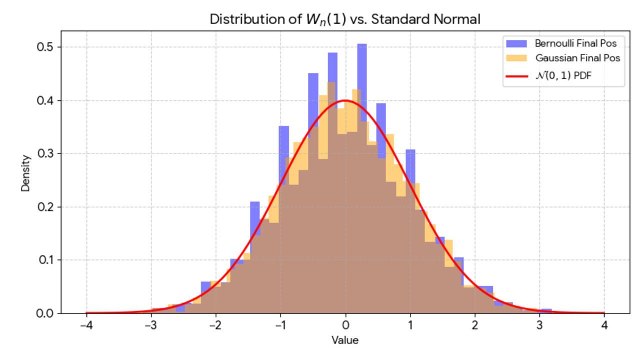

Figure 5: Empirical density of the final position $W_n(1)$ compared against the theoretical $\mathcal{N}(0, 1)$ PDF.

$$X_t = \mu t + \sigma B_t \tag{8}$$

where $\mu \in \mathbb{R}$ is the drift coefficient, $\sigma > 0$ is the diffusion coefficient (volatility), and B_t is a standard Wiener process.

11.1 Statistical Moments

The process X_t follows a normal distribution, denoted as $X_t \sim \mathcal{N}(\mu t, \sigma^2 t)$. Its moments are derived as follows:

- **Expectation (Mean):** By the linearity of expectation and the fact that $\mathbb{E}[B_t] = 0$:
$$\mathbb{E}[X_t] = \mathbb{E}[\mu t + \sigma B_t] = \mu t + \sigma \mathbb{E}[B_t] = \mu t$$

- **Variance:** Since μt is a deterministic constant, it does not contribute to the variance:

$$\text{Var}(X_t) = \text{Var}(\mu t + \sigma B_t) = \sigma^2 \text{Var}(B_t) = \sigma^2 t$$

- **Covariance:** For any two time points s and t, the covariance is scaled by the variance of the underlying Wiener process:

$$\text{Cov}(X_s, X_t) = \sigma^2 \min(s, t)$$

12 Ornstein-Uhlenbeck (OU) Process

The Ornstein-Uhlenbeck process is a stochastic process that tends to drift back towards a central mean. It is defined by the following Stochastic Differential Equation (SDE):

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t \tag{9}$$

where:

- θ **(Mean Reversion Speed):** Determines how quickly the process returns to the mean.
- μ **(Long-term Mean):** The equilibrium value that the process gravitates toward.
- σ **(Volatility):** The degree of randomness or fluctuation in the process.
- W_t**:** A standard Wiener process (Brownian Motion).

12.1 Analytical Solution

To solve the SDE, we rearrange the terms:

$$dX_t + \theta X_t dt = \theta\mu dt + \sigma dW_t$$

Multiplying both sides by the integrating factor $e^{\theta t}$:

$$e^{\theta t} dX_t + \theta X_t e^{\theta t} dt = \theta\mu e^{\theta t} dt + \sigma e^{\theta t} dW_t$$

The left side is the differential $d(e^{\theta t} X_t)$. Integrating from 0 to t:

$$\int_0^t d(e^{\theta s} X_s) = \int_0^t \theta\mu e^{\theta s} ds + \int_0^t \sigma e^{\theta s} dW_s$$

$$e^{\theta t} X_t - X_0 = \mu(e^{\theta t} - 1) + \sigma \int_0^t e^{\theta s} dW_s$$

Solving for X_t:

$$X_t = \mu + e^{-\theta t}(X_0 - \mu) + \sigma e^{-\theta t} \int_0^t e^{\theta s} dW_s$$

12.2 Mean and Variance

Mean: Since the expected value of the Itô integral is zero ($\mathbb{E}[\int_0^t e^{\theta s} dW_s] = 0$):

$$\mathbb{E}[X_t] = \mu + e^{-\theta t}(X_0 - \mu)$$

Note that as $t \to \infty$, $\mathbb{E}[X_t] \to \mu$.

Variance: Using **Itô Isometry**, $\mathbb{E}[(\int_0^t f(s) dW_s)^2] = \int_0^t f(s)^2 ds$:

$$\begin{aligned}
\mathrm{Var}(X_t) &= \mathbb{E}\left[\left(\sigma e^{-\theta t} \int_0^t e^{\theta s} dW_s\right)^2\right] \\
&= \sigma^2 e^{-2\theta t} \int_0^t e^{2\theta s} ds \\
&= \sigma^2 e^{-2\theta t} \left[\frac{e^{2\theta t} - 1}{2\theta}\right] \\
&= \frac{\sigma^2}{2\theta}(1 - e^{-2\theta t})
\end{aligned}$$

12.3 Numerical Simulation vs Analytical Solution

To computationally simulate the Ornstein-Uhlenbeck process, we apply the Euler-Maruyama discretization to the SDE:

$$X_i = X_{i-1} + \theta(\mu - X_{i-1})\Delta t + \sigma\sqrt{\Delta t} Z_i \tag{10}$$

where $Z_i \sim \mathcal{N}(0, 1)$ is a standard normal random variable.

Notice how the drift term $\theta(\mu - X_{i-1})\Delta t$ acts as a computational "rubber band." If X_{i-1} is far above the mean μ, the drift becomes negative, pulling the next step down. If X_{i-1} is below μ, the drift is positive, pulling it up.

12.3.1 Python Implementation

The following Python script simulates both the Euler-Maruyama approximation and the exact analytical step, allowing us to visualize the mean-reverting behavior.

Listing 3: Simulating the Ornstein-Uhlenbeck Process

```
import numpy as np
import matplotlib.pyplot as plt

T = 1.0          # Total time
N = 1000         # Number of steps
dt = T / N       # Time step size

theta = 5.0      # Speed of mean reversion
mu = 1.2         # Long-term mean
sigma = 0.3      # Volatility
```

```
X0 = 1.0        # Initial value

t = np.linspace(0, T, N)
X_em = np.zeros(N)
X_exact = np.zeros(N)
X_em[0] = X_exact[0] = X0

exact_var = (sigma**2 / (2 * theta)) * (1 - np.exp(-2 *
    theta * dt))
exact_std = np.sqrt(exact_var)

for i in range(1, N):
    Z = np.random.normal(0, 1)

    # Euler-Maruyama step
    X_em[i] = X_em[i-1] + theta * (mu - X_em[i-1]) * dt +
        sigma * np.sqrt(dt) * Z

    # Exact Analytical step
    exact_mean = mu + (X_exact[i-1] - mu) * np.exp(-theta *
        dt)
    X_exact[i] = exact_mean + exact_std * Z
```

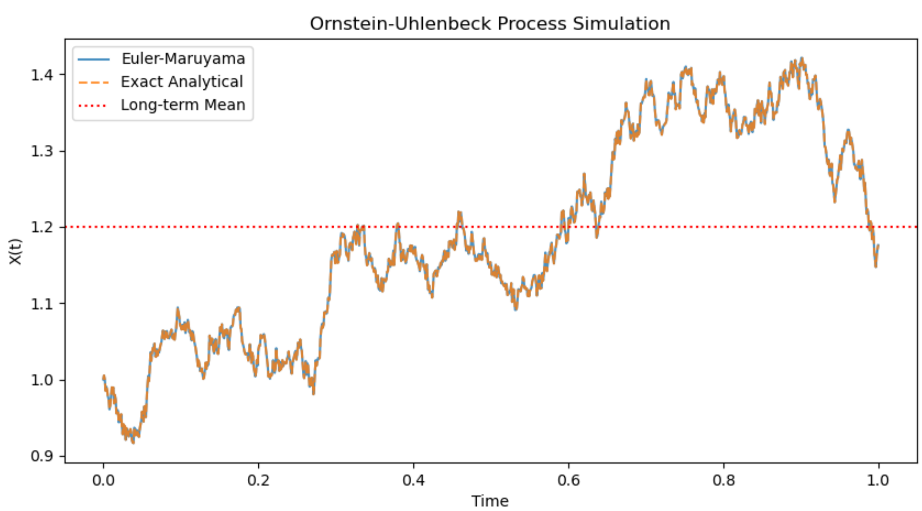

Figure 6: Numerical simulation of the Ornstein-Uhlenbeck process comparing Euler-Maruyama to the exact analytical step.

13 The Hotel Problem

The system models a hotel reception where guests arrive at random intervals and are served by a single receptionist. Guests are served in the order they arrive (FIFO). In this specific model, the **Queue Length** is defined as the total number of people currently at the counter, which includes the guest being served and those waiting behind them.

- λ: Mean arrival rate (average guests per unit time).
- μ: Mean service rate (average guests served per unit time).
- ρ: Traffic intensity or utilization factor ($\rho = \frac{\lambda}{\mu}$).
- n: Queue Length (total guests at the counter and in line).
- P_n: Probability that there are exactly n guests in the system.
- L: Average Queue Length (expected value of n).
- W_q: Average Waiting Time (time spent in line *before* service starts).

13.1 Steady-State Probabilities

For the system to be stable, we assume $\rho < 1$. Using the balance equations for a birth-death process, the probability of having n guests in the line is given by:

$$P_n = \rho^n P_0 \tag{11}$$

Since the sum of all probabilities must equal 1:

$$\sum_{n=0}^{\infty} P_n = 1 \implies P_0 \sum_{n=0}^{\infty} \rho^n = 1 \tag{12}$$

Using $\sum_{n=0}^{\infty} \rho^n = \frac{1}{1-\rho}$:

$$P_0 = 1 - \rho \tag{13}$$

Thus, the probability of a specific queue length n is:

$$P_n = (1 - \rho)\rho^n \tag{14}$$

13.2 Average Queue Length (L)

The average queue length L is the expected value of n:

$$L = E[n] = \sum_{n=0}^{\infty} nP_n \tag{15}$$

Substituting P_n:

$$L = (1-\rho)\sum_{n=0}^{\infty} n\rho^n \tag{16}$$

Using $\sum_{n=0}^{\infty} n\rho^n = \frac{\rho}{(1-\rho)^2}$:

$$L = (1-\rho)\frac{\rho}{(1-\rho)^2} = \frac{\rho}{1-\rho} \tag{17}$$

Substituting $\rho = \frac{\lambda}{\mu}$ yields:

$$L = \frac{\lambda}{\mu - \lambda} \tag{18}$$

13.3 Average Waiting Time (W_q)

To find the average time a guest spends waiting *before* being served, we first identify the average number of people strictly waiting ($L_{waiting}$), which is the total length L minus the person currently being served:

$$L_{waiting} = L - \rho = \frac{\rho^2}{1-\rho} \tag{19}$$

Applying Little's Law ($L_{waiting} = \lambda W_q$):

$$W_q = \frac{L_{waiting}}{\lambda} = \frac{1}{\lambda} \cdot \frac{\rho^2}{1-\rho} \tag{20}$$

Substituting $\rho = \frac{\lambda}{\mu}$:

$$W_q = \frac{\lambda}{\mu(\mu - \lambda)} \tag{21}$$

14 Derivation of Black-Scholes Model

I'll re-introduce a lot of concepts we've already explored, for the sake of continuity and re-iteration.

14.1 The Context

An option is a security which the right to buy or sell an asset within a specific interval of time, typically its bought for what we'll call the premium.

A call option is the kind of option that gives the right to buy a single share of common stock. A European option is a type of option which can only be accessed at a certain future time (that is, on the expiration date), at an exercise (striking) price at which the option can be bought.

So instead of buying a stock at a given time, options give the buyer the choice to buy the stock within an interval of time, so they can also choose to let the interval pass and the option expire.

Let's call the stock price S_0, the current price of the stock in the market. The exercise price X, and let's suppose there are T days left for expire.

For instance, let's say $S_0 = 1000$, $X = 1100$ and $T = 2$. If at $T = 2$, $S(2) = 1200$, you can pay X and pocket 100. If $S(2) = 900$, then you can choose not to pay X and all you lose relative to buying the stock at $T = 2$ is the premium paid to avail the option, which is usually not that high.

The goal of the model is to determine the option cost C_0 or the premium. The model also uses the market's volatility σ and the rate of interest on your money while you wait for T days (risk-free interest rate r).

14.2 Stochastic Processes for Modelling

A Weiner process W (standard Brownian motion) occurs under these conditions :

1. $W(0) = 0$
2. The function $t \to W(t)$ is continuous
3. The process has stationary independent normal increments.

   Stationary increments means that $W_{t+\alpha} - W_t$ has the same distribution for all α, so we can assume constant σ.

   Independent increments means that whatever happens before doesn't affect the future, or the random variables $W_{t_\alpha} - W_{t_\beta}$ are independent for all t_α and t_β (if the set is partitioned disjointly), or for $0 \leq s_1 < t_1 \leq s_2 < t_2 ... < s_n < t_n < \infty$, the variables $W_{t_i} - W_{s_i}$ are all independent jointly.

   With normal distribution, $W_{t+s} - W_s$ follows the normal distribution $N(0, t)$.

A variable follows a Weiner process with drift if it follows $dx = adt + bdW(t)$. Note that adt is deterministic and can be integrated regularly, but $bdW(t)$ represents the variability of the path of x as t changes. However, let's generalize to a non-linear model in itself, with X dependant on only t and W_t.

$$dX = a(t, X_t)dt + b(t, X_t)dW_t$$

$$X = X_0 + \int_0^t a(s, X_s)dt + \int_0^t b(s, X_s)dW_s$$

14.3 Derivation for Ito's Lemma

Let's consider only the second part for a second, because the first integral is quite easy. To get to Ito's Lemma to understand how to integrate dW_s, we need to also define a simple process. A simple process means that you hold the value constant between 2 timestamps say t_n and t_{n+1}, essentially a simple process follows a step type graph.

What this helps us define is the Riemann sum version of the integral, which allows us to say that $\int_0^t A_s dWs = Z_t$,, for A_s being a general simple process which is a function of time s.

Since $A(s)$ is a simple process, we can define $A(s) = Y_i$ for $s \in [t_i, t_{i+1})$.

$$Z_{t_j} = \sum_{i=0}^{j-1} Y_i[W_{t_{i+1}} - W_{t_i}]$$

We've used Z_{t_j} here just to stop at a particular change in the function $A(s)$. Generalizing to a general t,

$$Z_t = Z_{t_j} + Y_j[B_t - B_{t_j}]$$

t lying between t_j and t_{j+1}.

14.3.1 Derivation of First Lemma of Ito

$$f(W_t) = f(W_0) + \int_0^t f'(W_s)dW_s + 1/2\int_0^t f''(W_s)ds$$
$$df(W_t) = f'(W_t)dW_t + 1/2f''(W_t)dt$$

.

$$f(W_t) = f(W_0) - f(W_0) + f(W_{t/n}) - f(W_{t/n}) + f(W_{2t/n}) - f(W_{2t/n}) + ...f(W_{t(n-1)/n}) - f(W_{t(n-1)/n}) + f(W_t)$$

or

$$f(W_t) = f(W_0) + \sum_{j=1}^{n} f(W_{tj/n}) - f(W_{t(j-1)/n})$$

$$f(W_t) - f(W_0) = \sum_{j=1}^{n} f(W_{tj/n}) - f(W_{(t-1)j/n})$$

Via Taylor expansion :

$$f(W_{tj/n}) - f(W_{t(j-1)/n}) = f'(W_{t(j-1)/n}) * (W_{tj/n} - W_{t(j-1)/n}) + 1/2f''(W_{t(j-1)/n}) * (W_{tj/n} - W_{t(j-1)/n})^2 + O((W_{tj}$$

The first term as per our integral definition for a simple process is $\int_0^t f'(W_s)dW_s$. The second term simplifies via the following :

Quadratic Variation: $\lim_{n\to\infty}[W_{t_j} - W_{t_{j-1}}] = t$. This is simply a consequence of the variance being t.

Hence, $\lim_{n\to\infty} 1/2 \sum_{j=1}^{n} f''(W_{t_{j-1}})[W_{t_j} - W_{t_{j-1}}] = 1/2 \int_0^t f''(W_s)ds$.

The other terms go to 0 since they vary with $dsdW_s$ and d^2s etc. which all go to 0. This proves the first lemma.

14.3.2 Derivation of Ito's Second Lemma

If $f(t, X_t)$ is an Ito process with $dX_t = Z_t dt + y_t dW_t$, then

$$df(t) = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial X_t}Z(t)dt + \frac{\partial f}{\partial X_t}y_t dW_t + 1/2\frac{\partial^2 f}{\partial t^2}y^2(t)dt$$

This comes out of the Taylor series of $f(t, X_t)$ upto the quadratic term, where we won't drop the $(dW_t)^2$ term.

$$df = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial X_t}(Z_t dt + y_t dB_t) + 1/2\frac{\partial^2 f}{\partial X_t^2}(Z_t dt + y_t dW_t)^2$$

Expanding the quadratic term and neglecting dt^2 and $dtdW_t$ terms, we get :

$$df = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial X_t}(Z_t dt + y_t dB_t) + 1/2\frac{\partial^2 f}{\partial X_t^2}y_t^2 dW_t^2$$

14.4 Assumptions for the Black-Scholes Model

1. Constant rate of interest which is known to us (risk-free interest rate if we kept our money in a bond or somewhere else).
2. The stock pays no dividends or other distributions, so your only profits come from the option itself.
3. European option, so you cash out at the end only if you wish to.
4. There are no costs to sell or buy the stock or option, so you can buy and sell how many ever times you want without any transaction costs.
5. You're not restricted in terms of the minimum fraction to buy stocks, at that short-term interest rate.
6. There are no penalties to short-selling. Short-selling is when you borrow a stock with the bet that it falls, after which you sell it immediately. After it does fall, you buy it back and return the share to make a profit.

(Basically, you sell at old market price, buy at a new lower price, and pocket the difference). What the model assumes is that you're allowed to do this.

7. Log-normal distribution for stock price. What this means is that it's not the stock price that's varying in a Gaussian way, but the relative price increment that varies in a Gaussian way (which make sense, a higher valued stock would change with higher magnitudes). This also imposes positivity of S.

14.4.1 Log-Normal distributions

It makes sense that the normal increments compound multiplicatively rather than add, or that the log value of the price has these values adding up. Now via a charge of variable we'll examine these distributions.

If x = log(S) and x is normally distributed then S is log-normally distributed. Since the CDF will be equal upon variable change (probabilities remain conserved with variable change, so the CDF would stay the same, not the PDF).

$$F_S(s) = F_x(log(s)) \implies f_S(s) = f_x(log(s))/s$$

by chain rule.

Hence,

$$f(s) = \frac{1}{\sigma\sqrt{2\pi}s}e^{-\dfrac{(\log(s)-\mu)^2}{2\sigma^2}}$$

and the CDF is

$$F(s) = F(\frac{\log(s)-\mu}{\sigma})$$

.

Now to find the expected value that $S > K$, $y > \log(K)$, we compute the integral

$$L(K) = \int_K^\infty \frac{1}{\sigma\sqrt{2\pi}}e^{-\dfrac{(\log(S)-\mu)^2}{2\sigma^2}}dS$$

which gives us upon substitution :

$$\int_{\log(K)}^\infty e^y \frac{1}{\sigma\sqrt{2\pi}}e^{-\dfrac{(y-\mu)^2}{2\sigma^2}}dy$$

Simplifying this via completion of the perfect square gives us :

$$L(K) = e^{\mu+\sigma^2/2}F(\frac{\sigma^2+\mu-\log(K)}{\sigma})$$

Crucially,

$$dS = rSdt + \sigma SdW$$

. Hence, $\log S = f(S)$ gives via Ito's Lemma :

$$d(\log(S)) = (r - \sigma^2/2)dt + \sigma dW$$

This hence gives us after time T $\mu = \log(S_0) + (r - \sigma^2/2)T$ and variance $= \sigma^2 T$

14.5 Deriving Black Scholes Formula

$$C_0 = e^{-rT}E[\max(S_T - X, 0)] = \int_X^\infty (S_T - X)f(S_T)dS_T$$

This takes expectation of the function 0 till $S_T = X$ and then the difference after that.

The second integral directly gives just $e^{-rT}X[1 - N(-d_2)] = e^{-rT}XN(d_2)$, where N is the cumulative distribution function for normal distributions, and $d_2 = \frac{(\sigma^2 T + \log(S_0/X))}{\sigma\sqrt{T}}$.

The first integral from the log-normal distribution expected value math gives us $e^{-rT}e^{rT+\log S_0}N(d_1) = S_0 N(d_1)$ where $d_1 = \frac{\log(S_0/X)+(r+\sigma^2/2)T}{\sigma\sqrt{T}}$

Hence, $C_0 = S_0 N(d_1) - e^{-rT}XN(d_2)$.

14.6 Derivation of Black Scholes Equation

Let's design a portfolio with ϕ_t stocks and φ_t cash units, value $V = f$.

$$V = \phi_t S_t + \varphi_t$$

$$dV = (\varphi_t r + \mu S_t \phi_t)dt + \sigma S_t \phi_t dZ_t$$

Comparing with the Ito's Lemma equation gets us $\phi_t = \frac{\partial f}{\partial S_t}$ and $\varphi_t = f - \frac{\partial f}{\partial S_t} S_t$.

Plugging this in and comparing coefficients gives us :

$$\frac{\partial f}{\partial t} + rS_t \frac{\partial f}{\partial S_t} + 1/2\sigma^2 \frac{\partial^2 f}{\partial S_t^2} = rf$$

15 Euler–Maruyama Simulation and Monte Carlo Methods

While the Black–Scholes formula provides a closed-form analytical solution for pricing European call and put options, numerical simulation methods remain extremely important in computational finance. In practice, many financial derivatives such as Asian options, barrier options, and path-dependent contracts do not admit simple closed-form solutions. In such cases, Monte Carlo simulation becomes one of the most powerful and flexible tools for pricing.

The Black–Scholes stock price process follows the stochastic differential equation (SDE):

$$dS_t = \mu S_t dt + \sigma S_t dW_t \tag{22}$$

where S_t denotes the stock price at time t, μ is the drift parameter, σ is the volatility, and W_t represents standard Brownian motion.

To numerically approximate this stochastic process, we use the Euler–Maruyama method, which is the stochastic analogue of the classical Euler method for ordinary differential equations. It provides a discrete-time approximation of the continuous-time stock price dynamics.

Let the interval $[0, T]$ be divided into N equal steps of size:

$$\Delta t = \frac{T}{N} \tag{23}$$

Then the Wiener increment is approximated as:

$$\Delta W_n = \sqrt{\Delta t} Z_n \tag{24}$$

where $Z_n \sim \mathcal{N}(0, 1)$ are independent standard normal random variables.

Thus, the Euler–Maruyama discretization becomes:

$$S_{n+1} = S_n + \mu S_n \Delta t + \sigma S_n \sqrt{\Delta t} Z_n \tag{25}$$

This recursive relation allows us to simulate multiple possible future stock price paths.

15.1 Why Monte Carlo Simulation is Powerful

Monte Carlo simulation is especially valuable because it approximates the expected discounted payoff directly by generating a large number of possible future stock price paths. For a European call option with strike price K, the terminal payoff is $\max(S_T - K, 0)$, and the option price is estimated as:

$$C \approx e^{-rT} \frac{1}{M} \sum_{i=1}^{M} \max(S_T^{(i)} - K, 0) \tag{26}$$

where M is the number of simulated paths.

Monte Carlo methods offer several important advantages:

- They work even when no analytical solution exists.
- They are highly effective for pricing path-dependent derivatives.
- They handle high-dimensional problems better than finite-difference methods.
- Accuracy improves naturally as the number of simulations increases.
- They allow direct visualization of stochastic stock price behavior.

Although Monte Carlo methods can be computationally expensive, modern computing makes them highly practical and widely used in quantitative finance. Figure 7 illustrates several simulated stock price paths generated using the Euler–Maruyama method under the Black–Scholes framework.

Listing 4: Python code for Brownian Motion

```python
import numpy as np
import matplotlib.pyplot as plt

# Parameters
S0 = 100         # Initial stock price
mu = 0.08        # Drift
sigma = 0.2      # Volatility
T = 1            # Time to maturity (1 year)
N = 252          # Number of time steps
M = 20           # Number of simulated paths

dt = T / N
t = np.linspace(0, T, N + 1)

# Store all paths
paths = np.zeros((M, N + 1))
paths[:, 0] = S0

# Euler-Maruyama Simulation
for i in range(M):
    for j in range(N):
        Z = np.random.normal()
        paths[i, j + 1] = (
            paths[i, j]
            + mu * paths[i, j] * dt
            + sigma * paths[i, j] * np.sqrt(dt) * Z
        )

# Plot
plt.figure(figsize=(10, 6))

for i in range(M):
    plt.plot(t, paths[i])

plt.title("Simulated Stock Price Paths using Euler-Maruyama"
   )
plt.xlabel("Time")
plt.ylabel("Stock Price")
plt.grid(True)
plt.tight_layout()
plt.savefig("gbm_paths.png", dpi=300)
plt.show()
```

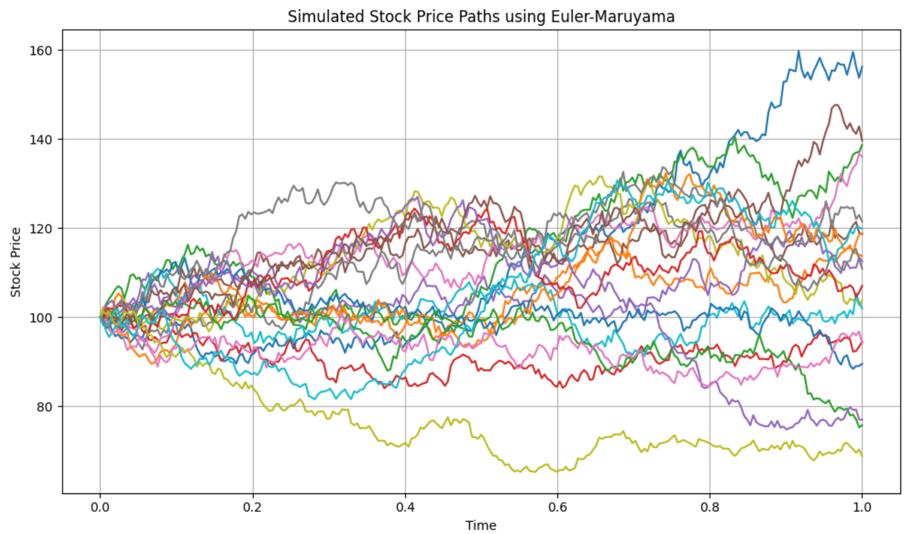

Figure 7: Simulated Geometric Brownian Motion paths using Euler–Maruyama approximation